Lecture 1 materials

Developing LLM applications on Databricks

This lecture connects local development, serverless execution, governed model access, and deployment. You will configure a Python project, call model services through Unity Gateway, ingest arXiv metadata, and deploy the ingestion notebook as a job.

Use these notes alongside the **lecture1** branch of the second-cohort repository. The course baseline is **Python 3.12, serverless environment 5, and Databricks Connect 18.0**. Examples follow the code available on 14 October 2026, including the local dependency updates.

Learning path

1. Serverless environments and cost attribution
2. Project configuration and dependencies
3. Local development and workspace execution
4. Foundation models and Unity Gateway
5. Calling the course model service
6. Provisioned throughput
7. External models and gateway controls
8. The arXiv ingestion notebook
9. Declarative Automation Bundles

Repository map

File	Purpose
notebooks/1.1_foundation_models_overview.py	Course model service and Responses API
notebooks/1.2_provisioned_throughput_deployment.py	Optional capacity deployment lab
notebooks/1.3_arxiv_data_ingestion.py	Metadata ingestion into a Delta table
notebooks/1.4_external_models_ai_gateway.py	Teacher demonstration of an external model

Model access, data access, and deployment permissions are separate. A successful CLI login establishes identity; the workspace and Unity Catalog grants determine which exercises that identity can run.

Read more Course repository

Serverless environments

Serverless compute is the course execution environment for notebooks and the ingestion job. Databricks manages the compute infrastructure. You choose an environment version and project dependencies rather than selecting a classic cluster runtime.

Environment 5 baseline

Setting	Course baseline
Python	3.12; environment 5 documents Python 3.12.3
Databricks Connect	18.0 locally, selected by the dev dependency group
Operating system	Ubuntu 24.04.3 LTS in the serverless environment
Project declaration	tool.databricks.environment in pyproject.toml
Job declaration	resources/process_data.yml → environments → spec

An environment version is a compatibility contract, not a claim that every installed package equals a classic runtime package. The project pins additional libraries. Check the active environment when debugging differences between your laptop and a notebook.

```
[tool.databricks.environment]
environment_version = "5"

[dependency-groups]
dev = ["databricks-connect~=18.0.0"]
```

Notebook dependencies

Choose environment 5 in the serverless notebook Environment panel. Install the project wheel built by the bundle for the ingestion job. Notebook 1.4 instead carries an inline environment declaration and a dependency on the parent project:

```
# /// script
# dependencies = [".."]
# [tool.databricks.environment]
# environment_version = "5"
# ///
```

Cost attribution

Serverless usage policies attach attribution tags to usage records. Select the assigned course policy for interactive work. The job uses `budget_policy_id: "${var.usage_policy_id}"`; the variable in `databricks.yml` must agree with the intended policy in `project_config.yml`. The application configuration does not automatically populate the bundle variable. Attribution is different from a spending cap.

Read more Environment 5 release notes · Serverless usage policies · Job environment

Project configuration and dependencies

`pyproject.toml` defines the `arxiv-curator` package, supported Python version, dependencies, build settings, and development tools. `project_config.yml` holds the workspace resource names used by the application.

Environment-specific application settings

The current dev configuration includes the following values. Use the resources assigned to your project when adapting the course.

```
dev:
  catalog: llmops_dev
  schema: arxiv
  volume: arxiv_files
  llm_endpoint: course_ops.gateway.chat
  embedding_endpoint: course_ops.gateway.embed
  warehouse_id: fed166457cfccb89
  experiment_path: /Shared/arxiv-curator-course
  lakebase_instance: lb-llmops-dev
```

`load_config()` calls `ProjectConfig.from_yaml()`. Pydantic validates required resource fields, and the loader adds the shared usage policy ID. The YAML key `schema` maps to `db_schema`, with a `schema` property for existing notebook code.

The current dev, acc, and prd blocks share `llmops_dev.arxiv`. Target names alone therefore do not isolate data. Assign separate catalogs or schemas before using this structure for independent production environments.

Dependency management

The local project pins `openai==2.14.0`, `databricks-sdk==0.118.0`, `pydantic==2.10.6`, and `mlflow==3.16.1`. Use the actual TOML and lockfile rather than copying an old dependency list from a handout.

```
uv sync --extra dev --group dev
uv run python --version
uv run python -m pip list
uv build
```

The `dev` extra supplies notebook and pre-commit tools. The separate `dev` dependency group supplies Databricks Connect. The lockfile records the resolved dependency set. Avoid adding standalone `pyspark` beside Databricks Connect.

Build and quality checks

Setuptools discovers `arxiv_curator*` under `src`, and reads the package version from `version.txt`. Ruff uses a 100-character line length and Python 3.12 rules. Install hooks with `uv run pre-commit install`; run them with `uv run pre-commit run --all-files`. The pytest settings point to `tests`, but this checkout has no test suite there.

Read more pyproject.toml · project_config.yml · Configuration loader

Local development and workspace execution

Authenticate with the CLI

Install the Databricks CLI for your operating system and authenticate using browser-based login. The bundle requires CLI 1.20.0 or later.

```
databricks --version
databricks auth login --host <workspace-url>
```

Authenticate to the workspace configured in the bundle target. A no-argument `WorkspaceClient()` uses configured SDK authentication; in a workspace notebook this can come from the notebook context. Remove stale environment overrides if a client resolves the wrong workspace or identity.

Run Spark from a local Python session

```
from databricks.connect import DatabricksSession

spark = (DatabricksSession.builder
         .serverless(True)
         .getOrCreate())
spark.read.table("samples.nyctaxi.trips").show(5)
```

Run this after syncing the project dependencies. Spark operations execute remotely. Python code outside Spark executes in the client process; `toPandas()` brings results into that process's memory. Use small results during local development.

VS Code workflow

1. Open the repository root and select its `.venv` interpreter.
2. Configure the Databricks extension for your workspace and the dev target.
3. Select serverless environment 5 and verify the connection.
4. Run a small cell before executing the ingestion notebook.

Notebook 1.3 uses `SparkSession.builder.getOrCreate()` in a Databricks notebook context. For an independent local script, use the explicit `DatabricksSession` setup above.

Getting code into the workspace

Workspace synchronization copies local files to a remote folder. A Git folder uses Git pull, commit, and push to exchange changes. Bundle deployment builds the wheel and deploys the declared job and project files together. Keep live-sync locations separate from bundle-managed locations, and make edits in the source repository before redeploying.

Source-format notebooks begin with `# Databricks notebook source`; `# COMMAND ----------` separates cells. Environment selection and installing the project are separate steps from authenticating.

Read more CLI authentication · Serverless Connect tutorial · VS Code configuration

Foundation models and Unity Gateway

Unity Gateway governs access to AI services through Unity Catalog. For this course, the application calls a named model service. That service selects the destination that performs inference.

Names and responsibilities

Object	Role in this repository
Model service	`course_ops.gateway.chat` is the application-facing name used by notebook 1.1.
Underlying model	The destination determines the model, capabilities, latency, and capacity. Do not infer it from the alias `chat`.
Model provider service	`course_ops.gateway.openai` defines the external provider connection and its permitted target.
Provisioned serving endpoint	Notebook 1.2 allocates dedicated model capacity, then creates a model service that routes to it.

A model service can route to pay-per-token models, provisioned capacity, or external providers. Its name has the form `catalog.schema.name`. Inspect the service in Catalog Explorer or AI Gateway to see the actual destination and routing configuration.

Model capabilities

Choose a model based on the task and supported API. Text generation supports conversational or document workflows; embeddings represent inputs for similarity search; vision-capable models accept images; reasoning-capable models may spend additional computation on complex tasks. These capabilities overlap. A vision input capability does not imply image generation, and a reasoning model does not guarantee correct answers.

Access and request format

Calling a course service requires `USE CATALOG`, `USE SCHEMA`, and `EXECUTE` on the service. Managing a service or its provider requires additional privileges.

The unified gateway base URL is `https://<workspace-host>/ai-gateway/mlflow/v1`. The request's `model` value is the model service name. This differs from older examples that use `/serving-endpoints` and a serving endpoint name.

Databricks recommends the Unified Responses API for new provider-independent requests. Notebook 1.1 uses Responses; notebook 1.4 uses Chat Completions. Supported request fields and modalities still depend on the destination.

Read more Unity Gateway overview · Custom model services · Query model services

Calling the course model service

Notebook 1.1 resolves the workspace through the SDK, creates a short-lived token, and passes it to the OpenAI-compatible client. The following compact example follows its authentication and gateway URL construction.

```
from databricks.sdk import WorkspaceClient
from openai import OpenAI

workspace_client = WorkspaceClient()
databricks_token = workspace_client.tokens.create(
    lifetime_seconds=1200
).token_value

client = OpenAI(
    api_key=databricks_token,
    base_url=f"{workspace_client.config.host}/ai-gateway/mlflow/v1",
)
response = client.responses.create(
    model="course_ops.gateway.chat",
    max_output_tokens=256,
    input="What is Databricks?",
)
print(response.output_text)
```

The token expires after 20 minutes. This path requires permission to create personal access tokens, as well as permission to call the service. Create a new token and client after expiry. Do not print the token or save it in notebook output.

The credentials authenticate to Databricks. An OpenAI provider API key is not required for the caller of the course service. The SDK host supplies the URL directly, with no separate hard-coded workspace constant.

Conversation and usage

The full notebook passes prior user and assistant messages as the Responses input. It reads `response.output_text` and, when usage is present, reports `total_tokens`, `input_tokens`, and `output_tokens`. A token limit bounds the response length; it does not validate the answer.

Diagnosing a failed call

Symptom	Check
Authentication or token error	Selected SDK identity, token-creation policy, token expiry, and workspace host.
Permission denied	Parent catalog/schema access and service EXECUTE privileges.
Invalid model or request	Fully qualified service name, routing destination, and supported API fields.
Rate limit or backend failure	Service and provider limits, allocated capacity, and configured fallback behavior.

Read more Notebook 1.1 · Unified API reference

Provisioned throughput

Notebook 1.2 is an optional deployment lab. It uses `system.ai.gpt-oss-120b`, checks model-unit eligibility, creates an on-demand endpoint, and exposes it through `course_ops.gateway.provisioned_chat`. It does not change the shared `course_ops.gateway.chat` service.

Capacity choices

Option	Planning consideration
Pay-per-token	Shared capacity for exploratory or variable use. Check model availability and request limits.
On-demand provisioned throughput	Allocated capacity without a fixed term. The lab uses model units and charges continue while capacity remains allocated.
Reserved provisioned throughput	A separate capacity commitment for a fixed term. Supported models and lifecycle restrictions differ.

Reserved capacity currently offers 1- or 3-month terms. Do not treat an on-demand SDK creation call as a reservation purchase. Overflow beyond a reservation is rate-limited unless a model-service fallback is configured.

Lab sequence

1. Review resource names, source model/version, and the workspace URL in notebook 1.2.
2. Keep `RUN_DEPLOYMENT` and `RUN_INFERENCE` false while reading the cost calculation.
3. When ready, enable deployment. The optimization lookup must report eligibility and a positive `model_unit_chunk_size`.
4. The requested allocation is `CAPACITY_CHUNKS × model_unit_chunk_size`. The default chunk count is two.
5. After the endpoint and model service are ready, enable inference.
6. Enable `RUN_CLEANUP` in the cleanup cell to delete the lab service and then its endpoint.

Sizing and cost

Estimate requests per minute, input and output lengths, burstiness, and acceptable latency. For example, 60 requests per minute with 2,000 input and 300 output tokens represent 120,000 input tokens and 18,000 output tokens per minute. This is workload arithmetic, not a guarantee of latency or a conversion to model units.

The notebook's fixed-allocation estimator multiplies the allocation in 50-unit blocks by hours and a DBU rate. Treat that rate as a lab assumption and check current pricing before deploying. DBUs must be converted using your contracted price, with other charges considered separately. Benchmark realistic prompts before choosing production capacity.

Read more Notebook 1.2 · On-demand capacity · Reserved capacity

External models and gateway controls

Notebook 1.4 calls `course_ops.gateway.openai_demo` through the same gateway URL. Its provider service references the Unity Catalog secret `course_ops.secrets.openai_api_key` and targets `gpt-4o-mini`. The provider credential remains behind the gateway.

```
workspace = WorkspaceClient()
client = OpenAI(
    base_url=f"{workspace.config.host}/ai-gateway/mlflow/v1",
    api_key=workspace.config.authenticate()["Authorization"]
        .removeprefix("Bearer "),
)
response = client.chat.completions.create(
    model="course_ops.gateway.openai_demo",
    messages=[{"role": "user", "content": "What is an AI gateway?"}],
    max_tokens=256,
)
print(response.choices[0].message.content)
```

Import `WorkspaceClient` and `OpenAI` as in the previous example. This notebook reuses the SDK's current bearer credential; it does not create a new token. That differs from notebook 1.1. The demo grants service access to `grp-teachers-llmops`; a student permission error may be expected.

Routing and fallbacks

Traffic percentages choose the initial destination. Fallbacks retry a failed attempt against backups in a configured priority order; traffic weights are not rerun for retries. Evaluate every backup for response quality and API compatibility, as well as availability. Session affinity can keep identified sessions on one destination.

Guardrails and observability

Unity Catalog grants determine access. Service policies evaluate request and response content; this feature is currently Beta. A policy denial can arrive as an HTTP 200 response with policy details, so status codes alone do not prove that the model answered successfully.

Usage monitoring and payload logging serve different purposes. `system.ai_gateway.usage` records request activity; inference tables require their own logging configuration and permissions. Verify routing outcomes and usage after a test call.

Deployment boundary: `resources/gateway_demo.yml` contains the provider/service definitions and a 400-requests-per-minute service limit, but `databricks.yml` currently includes only `resources/process_data.yml`. A normal bundle deploy does not create the gateway demo. Its provisioning is a teacher-managed prerequisite.

Read more Notebook 1.4 · Model providers · Routing · Service policies · Usage

The arXiv ingestion notebook

Notebook 1.3 is the data exercise. It fetches recent AI and machine-learning paper metadata and stores a small sample in Unity Catalog. It does not download or parse PDFs in this lecture.

Configuration and execution

```
from pyspark.sql import SparkSession
from arxiv_curator.config import get_env, load_config

spark = SparkSession.builder.getOrCreate()
env = get_env(spark)
cfg = load_config("../project_config.yml", env)

CATALOG = cfg.catalog
SCHEMA = cfg.schema
TABLE_NAME = "arxiv_papers"
```

The job passes its bundle target as the `env` notebook parameter. `get_env()` reads that widget and falls back to `dev` if it cannot retrieve it. The loader validates the selected configuration before the notebook creates the schema if needed.

Data contract

`fetch_arxiv_papers()` defaults to 100 results, but the demonstration call explicitly asks for 10. It searches `cat:cs.AI OR cat:cs.LG`, ordered by submission date. The result includes paper IDs, titles, author arrays, abstracts, dates, categories, and PDF URLs.

The explicit Spark schema stores `published` as a long integer formatted `YYYYMMDDHHMM`, and `authors` as an array of strings. `processed` and `volume_path` are left null for later processing. Keeping this contract stable makes subsequent notebooks easier to connect.

```
table_path = f"{CATALOG}.{SCHEMA}.{TABLE_NAME}"
df.write.format("delta").mode("overwrite").option(
    "mergeSchema", "true"
).saveAsTable(table_path)
```

Overwrite behavior: rerunning the demonstration replaces the target table's contents. Use the assigned development schema. A production ingestion job needs a deliberate incremental strategy and duplicate handling.

Verification

Read the table back, inspect its schema and row count, and display a few paper IDs and titles. The notebook then groups by primary category and shows recent papers. Its sample logging assumes at least one API result; an empty result needs handling before indexing `papers[0]`.

`DataProcessor` in the package has additional PDF download, parsing, and chunking methods. Those methods are separate from the lecture 1.3 metadata-only flow; they are not executed by the current ingestion job.

Read more Notebook 1.3 · Later processing utilities

Declarative Automation Bundles

Declarative Automation Bundles, formerly Databricks Asset Bundles, package code, build artifacts, and resource configuration for deployment. The bundle entry point is `databricks.yml` in the repository root.

This repository uses `engine: direct`, builds a wheel with `uv build`, and includes the ingestion job definition. The dev target is the default and uses development mode. The acc and prd targets use production mode. Review the resolved target before deploying; target settings and application data locations are separate.

The deployed ingestion job

```
# resources/process_data.yml, excerpt
environments:
  - environment_key: default
    spec:
      environment_version: "5"
      dependencies:
        - ../dist/*.whl
tasks:
  - task_key: arxiv_data_ingestion
    environment_key: default
    notebook_task:
      notebook_path: ../notebooks/1.3_arxiv_data_ingestion.py
      base_parameters:
        env: "${bundle.target}"
```

The resource key is `data-pipeline`; its display name is `arxiv-data-pipeline`. Its schedule is 06:00 Europe/Amsterdam. The pause variable defaults to PAUSED, acc remains paused, and prd sets UNPAUSED. Deploying prd can therefore activate scheduled execution.

```
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev data-pipeline
databricks bundle destroy -t dev
```

Validation checks the bundle configuration; it does not prove that an API call or notebook will succeed. Deployment builds and uploads the artifact and configures resources. Running executes the named job. Destruction removes bundle-managed resources and should be deliberate.

Read more Bundle documentation · databricks.yml · resources/process_data.yml